SCORE: A Framework for Synthesizing Complete, Owned, Responsive Environments

J.P. Kardolus, Babon Innovations B.V., Utrecht, The Netherlands

Working draft, 13 October 2026

Abstract

Games, films and robot training need 3D worlds that are complete, editable and owned by the people who make them. World models and one-shot 3D generators can produce a convincing place from a sentence or a picture, but they give frames that no engine can open, or one fused scene without separate objects or clear rights. Studios have always built worlds in stages, from concept art and layout to modelling, surfacing, lighting, sound and review. We present SCORE, a world generation framework that keeps these stages and automates them with open vision foundation models, procedural tools and coding agents, while the creator chooses and reviews. Every stage is a node of one runner: its output is keyed on its inputs and its code, judged by gates that answer pass, fail or unknown, and accepted only when nothing got worse, so a fault is fixed in its own stage and only what follows it runs again. Paid steps are cleared against a month's budget read from the provider's own bill. Every model in the framework allows commercial use of its output, so the creator stays the author of the result. We describe the framework as built for its first world, the game world of 2099 with 18 places, and the measured gates of each of its layers: surfaces, resting, light, motion, effects, sound and people. Results per place are being gathered. Code and paper are available at https://github.com/Babon-Innovations-b-v/score (MIT license).

Gap: Figure 1, four worlds. World 1's panel: the square (the prologue on Earth), the hub (the Moon base) and the Mars camp, each concept beside its finished stage rendered from the concept's own camera; worlds 2 to 4 shown as not started. It waits on those three places passing their completion gate on their final assembly; the square and the camp wait on their people's bodies.

1 Introduction

Interactive 3D worlds are the raw material of games, film and embodied AI [1], and building one is expensive: every object has to be modelled, surfaced, placed, given collision, lit and given sound in one style, and the world has to stay editable. Generative models promise to take over much of this, but in GDC's 2026 survey 52 percent of game workers thought generative AI was hurting the industry [2]; a tool for building worlds has to leave the result in the creator's hands.

World models generate a world as frames [3], [4] that no engine can open. 3D world generators [5], [6] produce geometry, but mostly as one fused asset, from which separate objects are recovered after the fact [7], [8]. Both work in one shot, so a creator cannot find or change the step behind a fault. Procedural systems [9], [10] and coding agents that write scenes as programs [11], [12], [13], [14] keep the world explicit, but rebuild a scene that already exists or draw their objects from a fixed library, and such agents judge their own geometry unreliably [15]. Open vision foundation models [16] now read shapes [17], masks [18], depth [19] and parts [20] out of pictures, and a coding agent

can connect them as tools; none of them provides one style across objects that different models made, which the field names as an open problem [21], [22].

Studios build worlds in stages, each with an output a person can check. SCORE keeps these stages and automates them with the models above, procedural tools and coding agents, while the creator chooses and reviews. Its worlds are **complete**: every object, surface, light and sound exists wherever the player looks. They are **owned**: every model in the framework allows commercial use of its output, so the creator stays the author, holding the full commercial rights. They are **responsive**: every object has real collision, mass and material. And they can be any kind of **environment**. To our knowledge, no other openly available world generation work joins vision foundation models into engine-ready scenes that are built from reference pictures, steered by a person's choices at each step, and made only with models that allow commercial use of their output.

This paper makes three contributions:

- **The SCORE framework:** stages as nodes of one runner that keys, judges and accepts every output, over one canonical scene: a generated base under the creator's edit layer (Section 2).
- **Stages with measured gates** for every layer of a place: models, surfaces, resting, light, motion, effects, sound and people, and a completion gate over all of them (Section 3).
- **Evidence from a first world**, with its spend per stage read from the ledgers and the bill (Section 4).

2 The framework

A world is a table of stages; world 1's has 95 rows. A **node** is one stage for one place (`assembly@hangar`) or for the whole world, and one runner makes, judges and accepts every node. The creator's picks (the concept of each place, the style, the references) are the inputs; from there everything runs without a typed yes, and the creator reviews.

2.1 Keys, gates and accepts

Keys. A node is made only when its key has no finish record. The key hashes the stage and place, a code version, the contracts of its output and inputs, its parameters, a seed derived from the world's, the pinned model images, every world file it reads and the accepted output of every node it reads. The code version hashes every source file the stage's make and gates import, walked through their imports; the runner and the renting code are left out, since they change what a run costs, not what it makes. A fault fixed in one stage moves the keys of that stage and of what follows it, and nothing else. Outputs are kept by content hash, never overwritten, each stamped with its schema in metres, seconds and degrees.

Gates and accepts. Each gate answers pass, fail or unknown, with measures and a reason; missing evidence is unknown, and unknown blocks like fail. The runner accepts an output by itself when every gate passed and it is no worse than the output accepted before: no answer dropped, no measure moved the wrong way, no gate went missing. Later nodes read only accepted outputs. The creator keeps four commands, from his own terminal: reject an accepted output (it moves back to the one before, and every node that read it is stale), accept a worse one with his reason, waive one gate for one key, and override the runner for one paid run. None is needed for a run. Agents are kept from them by a barrier, not a lock: the commands want a terminal, the store refuses the creator's records from any process an agent started, and a hook blocks the other routes.

Spending and review. A paid node is cleared only when every gate on its inputs passed and the month's ceiling has room for its estimate (Section 3.7); every place in the code that rents a machine or calls a paid model refuses without that ticket, and a run that would buy again more than EUR 5 of accepted outputs needs a written reason. Each node's review page is built from the store alone (its gates' answers and measures, who accepted it, the command that rejects it), and the creator reviews a place in a USD viewer on the store's copy of its accepted assembly; nothing is rendered for review.

Gap: Figure 2, the stages on the hub (concept, plan, inventory, close-ups, a library surface at three wears, the assembled room), **and Figure 3, the greenhouse's review page.** They wait on the hub's and the greenhouse's one assembly passing its completion gate; the earlier hub figure predates the runner and is withdrawn.

2.2 From concept to scene

Inventory, plan and record. A coding agent lists every visible element of the concept as an inventory row (kind, anchor, size, count) or drops it with a written reason; nothing is made or placed that is not a row. It draws a dimensioned plan, gated on `mapped` (every element of the concept a row, a door or a written drop; every door says where it leads). The scene record is written from the plan, inventory, made ground and sky, gated on `game-free` (nothing copied from the game the world came from), `plan-matched` (the shell within 5 cm of the plan), `openings` (every doorway onto a floor, no view into nothing) and `seed-matched` (every old entry matched or dropped with a reason).

The canonical scene. Every place is one OpenUSD stage [23] in which each object carries its kind, transform, collision and the library surface of each part. Its layers lie in a fixed order, the creator's edit layer on top, then people, sound, light, motion, effects, crops, machines, settle, ground and the generated base, so an edit survives a regenerated base. Each place is assembled once, from every layer's accepted output; its `layers` gate answers unknown while a layer the place needs has none, and an old output never stands in.

Models, generation first. Every object goes the prop route first: Nano Banana Pro [24] draws its close-up from its crop of the concept, Pixal3D [17] builds it, the surface stage paints it, and a free likeness gate compares its outline from eight sides with its close-up. Only an object whose measured checks fail is built by a code builder that a coding agent writes. This reverses the earlier parts check, which built a kind in code whenever its code build showed every part of the close-up (the creator's choice, 12 October 2026). Every real-world object stays one model: a console is a desk carrying separate monitors. Most of world 1's models predate the change and enter as unverified seeds, judged by the stages after them.

Gap: Figure 4, one scene, an edit kept (the wreck from camera 1, with an edit that survives a regenerated base whose models were repainted), **and Figure 5, generation first** (four of the hub's objects, two generated and two falling back to code, with passes, euros and agent hours for both orders). Figure 4 waits on the wreck's surface check and assembly; the hub's comparison has run and waits on its record being published.

3 Stages and their gates

3.1 Surfaces: the finish map

Models give shape only; every part takes one surface from a shared library of rule-based surfaces, written as ProcFunc functions [25] on Infinigen's shaders [9], coloured from the place's palette and worn only where wear has a cause (Appendix D). Gemini 3.1 Pro [26] lists the close-up's finishes and names a library surface for each. Nano Banana Pro draws the close-up again with one flat colour per finish, up to three times until the map fits the close-up's outline, then draws unlit renders of the raw model's sides and back against that map. SegviGen [27] splits the mesh guided by the close-up's map, every map votes on the faces its view sees, and a part whose second finish holds a fifth of it fails. Each part is painted from its library surface and baked; the picture's own colours are never kept, because when every piece kept them, every piece brought its own rust and stains (Appendix A).

The bake is checked against the close-up. A finish's surface must lie within ΔE 32 of its colour there, else a nearer surface of its family is taken or a new one is proposed to the creator, and the part fails by name. Two checks came from the greenhouse's water tank, which the creator found baked as three near greys and standing tipped: a bake fails when it turns a finish the close-up shows lighter into a darker one, or all its surfaces into one grey the close-up does not have; and a model on legs has its short legs stretched onto one level plane before its bake, a foot more than 3 cm off the floor failing. Each model is dirtied as much as its close-up shows, capped at its place's dirt.

Gap: Figure 6, shape and surface apart. Library swatches; the water tank's close-up, finish map, split and painted model; the hub's tool board with picture colours against library paint. It waits on the surface line at the greenhouse's and the hub's final assemblies.

3.2 Resting

Loose objects are dropped onto the made ground in a physics simulation; one that would turn more than 15 degrees or drift more than 30 cm is a resting fault of the layout and keeps its laid pose. The settle stage lays where each object comes to rest as a layer of its own (held objects snapped onto their holders, a stack onto one support, an object that would rest worse kept as laid). Its gates are `rested` and `resting` (no fault that appears with the layer and not without it).

3.3 Light

Lamp strengths are measured, never copied from the source game. The light layer places live lamps where the scene record puts them, with the sky, the sun and, seen from the Moon, the Earth. One paid render per place gives every lamp its own Cycles light group on the place's views; because light adds up, the look at any strengths is mixed from these passes without rendering again, and a free sweep chooses the scale that brings the place's brightness within 1.5 times its concept's, halving any lamp that turns what it lights white. Under the open sky the suns are swept after the lamps. A lamp is measured where it shines: one whose fixture stops 90 percent of its rays moves out of the fixture, and one behind a clear lens stands just outside it, since Cycles passes no light through a lens that also glows. Baking a pass per lamp, mixed live in the engine, is planned.

3.4 Motion, effects and sound

Motion. Each moving thing is a mover in the place's record (a path, a hinge, a spin, a rail, a turn, a climb) with its kind's speed range cited from real machines. The gates read the composed

stage every quarter second: `clear` (no mover within 3 cm of a still thing or another mover), `speeds`, `loops`, `grounded` and `covered` (every prim of a moving kind that stays still says why).

Effects are emitters on a prim, measured against real footage: the launch's cloud and flame in rocket heights from photographs of real lift-offs, the camp's dust devils from rover films of Mars. The gates are `attached`, `footage` (every measure of the baked particles within the range read off footage), `light` (the flame lights the pad at least twice as brightly as the place's lamps) and `outside` (no open-sky particle inside a room).

Sound. Every source a place names (room tones, objects, motions, each surface's steps and impacts) gets the world's picked file or the best of eight MOSS-SoundEffect [28] takes of its brief, gated on `sources`, `level` (loudness, peak and length), `likeness` (CLAP [29] score at least 0.30, the tenth percentile of the world's picks) and `no freesound`.

Gap: Figure 7, light (the old station with the Earth lit, a dark side away from the sun; one room at its swept strengths beside its concept), **and Figure 8, things that move** (the launch's lift-off, a rover leaving the garage with wheel dust, the camp's dust devil, an airlock cycle). They wait on those places' assemblies; the rover's drive waits on its painted model.

3.5 People

People are made like props. A close-up of the dressed person in SOMA-X's bind pose, drawn by FLUX.2 klein [30], is built by Pixal3D and painted by the surface line. SOMA-X's body is fitted just inside the look (Gauss-Newton on 24 shape numbers), and every point of the look takes the skin weights of its closest body triangle, so Kimodo's clips [31], written from sentences, move it. The **shell gate** passes a look when 99 percent of its soft points lie within 5 cm of the body, its worst point recorded. Hard parts ride one joint each (hair, boots, bare hands, helmets), and belts are worn as cloth. Each clip gets its own verdict (tucks, hard parts stretched, cloth through cloth, body through cloth), and a cast uses only the clips that pass. A look seen only far away is judged on its far body, and a look's colour comes from the cast record's words (Appendix B).

Casts. SAM 3 [18] finds the figures on a place's views, and a vision-language judge tells a person from a picture, a reflection or an empty suit. A cast is gated on `counted`, `standing`, `clear`, `made` and `drapes`; a crowd stands only where no copy fills a tenth of a view's height, and every person wears their look's accepted body, never a stand-in.

Gap: Figure 9, people made like props. The player from close-up to a walking frame, and the square's crowd of 10,000 with its front row. It waits on the people batch; the player's work suit had no usable clip because of its belt.

3.6 Completion

A place is complete when its completion gate passes on its final assembly: every inventory row has its object or a written reason; nothing visible is unmade or a code placeholder; no resting fault; every pick of the creator is the model shown; and the place holds what it needs: its cast in made looks, its effects and motions, its crops, every generated model held to its close-up and repainted, every lamp and room tone of its record, every place it shows, and nothing copied from the source game. Each answer is tied to a fingerprint of the stage; an answer made on another stage is unknown, and a hook refuses to end an agent's session that claims a place is done while its gate fails. Coding agents judging their own scenes agree with a measured score on geometry only 45.8 percent of the time [15], so no agent's word completes a place.

Gap: Figure 10, responsive. A room in a physics engine, a dropped crate and a door reacting. The physics stage writes UsdPhysics collision, mass and friction and runs a Bullet trial, so far on the habitat only; until it passes, responsiveness is that trial.

3.7 Budget and ledgers

Every rented machine writes a ledger line with its euros, and every call to Google's models writes a started line before the call and a priced line after it, so a call that never returns still counts. Each line carries the ticket of its node. The month's spend is the higher of the ledgers and the provider's bill, because the ledgers missed costs: on 12 October 2026 the month's cloud bill stood at EUR 1,554 against EUR 1,387 in the ledger (disks and addresses billed by the hour, storage, some card time), and on 11 October Google billed EUR 549 for a day in which the earlier ledger held EUR 30. Over 13.8 hours of world 1's run from the first ticketed line, the ledgers hold EUR 117.45 of rented machines and EUR 34.19 of Google's models; the largest stages were painting (EUR 37.61), the finish maps (EUR 26.59), the people's bodies (EUR 16.09) and the light passes (EUR 10.17).

4 Evidence

We build four worlds with the same stages: the game world of 2099 (a prologue on Earth, a Moon base and a camp on Mars), a stylised underwater world, a simulation world for robots, and a third-person fantasy world. World 1 has 18 places, each its own line of the table, assembled and judged complete once, when every layer it needs stands accepted.

Gap: Table 1, results per place and stage, and the film. Per place: inventory rows, models generated and built in code, the layers present, faults caught before and after spending, time, euros from Scaleway's and Google's bills, and the creator's verdict; and a four-minute walk through world 1 from each place's own cameras with its sound. They wait on each place's one assembly and completion gate.

With and without the agent tools. On two of three places of world 1, an agent with the completion gate and its tools used about half the output and cost of one without them; on the third, the one without left fewer faults. Each arm ran once (Appendix E).

5 Limitations

Every verdict is one creator's, and he accepted one place, the hub, before the runner existed. Closed, paid models draw the concepts, the prop route's close-ups and the finish maps (Nano Banana Pro) and list the finishes (Gemini 3.1 Pro); their limits set the pace, and they can change price or be retired, which limits reproducibility. People are not finished: belts worn as hard parts could not follow a deep forward bend. Light is live, not baked, and was measured on earlier layouts; objects are physical but not interactive.

6 Future work

The three further worlds; an engine adapter with mass and friction; interactable objects, which robotics needs; baked per-lamp light; open models for close-ups and finish maps; a shared library of locked, versioned parts that worlds reference instead of buying again; LEGO-Bench [15]; and a measure of whether a place keeps one style.

7 Conclusion

SCORE builds worlds from a creator's choices in stages whose outputs are kept, keyed and judged, so a fault is fixed where it happens and only what follows it runs again. On its first world, every layer of a place, from surfaces to light and people, has gates measured against the concept or real footage. Whether the framework holds across a whole world, and across kinds of worlds, is what world 1's completed places and the further worlds will show.

References

[1] Y. Yang *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[2] D. Argüello, "36 percent of game workers use generative AI, but half think it's bad for the industry." Game Developer, Feb. 2026.

[3] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, *et al.*, "Genie: Generative interactive environments," *arXiv preprint arXiv:2402.15391*, 2024.

[4] J. Parker-Holder and S. Fruchter, "Genie 3: A new frontier for world models." Google DeepMind blog, Aug. 2025. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/

[5] D. Wang, H. Jung, T. Monnier, K. Sohn, *et al.*, "WorldGen: From text to traversable and interactive 3D worlds," *arXiv preprint arXiv:2511.16825*, 2025.

[6] Tencent Hunyuan, "HY-World 2.0." GitHub repository, 2026. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0

[7] J. Hu, J. Guo, J. Cen, C. Yang, S. Li, and W. Shen, "WorldAct: Activating monolithic 3D worlds into interactive-ready object-centric scenes," *arXiv preprint arXiv:2605.15843*, 2026.

[8] M. Niu, J. He, R. Yu, L. Fu, *et al.*, "WorldSculpt: Generating compositional worlds from grounded videos," *arXiv preprint arXiv:2609.05416*, 2026.

[9] A. Raistrick *et al.*, "Infinite photorealistic worlds using procedural generation," *arXiv preprint arXiv:2306.09310*, 2023.

[10] A. Raistrick *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, 2024.

[11] Z. Hu, A. Iscen, A. Jain, T. Kipf, *et al.*, "SceneCraft: An LLM agent for synthesizing 3D scene as Blender code," in *International conference on machine learning (ICML)*, 2024.

[12] Z. Li, Y. Liao, and B. Zhu, "Recursive code world models: Building complex worlds through recursive scene programs," *arXiv preprint arXiv:2609.11499*, 2026.

[13] C. Guo, J. Li, Y. Li, and Z. Huang, "WorldClaw: Agentic 3D open-world generation at scale," *arXiv preprint arXiv:2608.05248*, 2026.

[14] L. Yin, W. Cheng, Z. Qin, T. Huang, *et al.*, "AutoUE: Automated generation of 3D games in Unreal Engine via multi-agent systems," *arXiv preprint arXiv:2603.07106*, 2026.

[15] X. Li *et al.*, "LEGO-Anything: Coding agents for 3D scene reconstruction," *arXiv preprint arXiv:2609.36380*, 2026.

[16] M. Oquab, T. Darcet, T. Moutakanni, *et al.*, "DINOv2: Learning robust visual features without supervision," *arXiv preprint arXiv:2304.07193*, 2023.

[17] D.-Y. Li *et al.*, "Pixal3D: Pixel-aligned 3D generation from images," *arXiv preprint arXiv:2605.10922*, 2026.

[18] N. Carion *et al.*, "SAM 3: Segment Anything with concepts," *arXiv preprint arXiv:2511.16719*, 2025.

[19] R. Wang *et al.*, "MoGe-2: Accurate monocular geometry with metric scale and sharp details," *arXiv preprint arXiv:2507.02546*, 2025.

[20] Y. Lin *et al.*, "PartCrafter: Structured 3D mesh generation via compositional latent diffusion transformers," in *Advances in neural information processing systems (NeurIPS)*, 2025.

[21] J. Wu, Z. Lou, J. Liu, D. Du, C. Guo, and S. Guo, "From visual synthesis to interactive worlds: Toward production-ready 3D asset generation," *arXiv preprint arXiv:2604.23629*, 2026.

[22] Z. Yang *et al.*, "FlowScene: Style-consistent indoor scene generation with multimodal graph rectified flow," *arXiv preprint arXiv:2603.19598*, 2026.

[23] Pixar Animation Studios and Alliance for OpenUSD, "OpenUSD: Universal scene description." 2026. Available: https://openusd.org/

[24] Google, "Gemini 3 Pro Image ('Nano Banana Pro') through the Gemini API." 2026. Available: https://ai.google.dev/

[25] A. Raistrick *et al.*, "ProcFunc: Function-oriented abstractions for procedural 3D generation in Python," *arXiv preprint arXiv:2604.26943*, 2026.

[26] Google, "Gemini 3.1 Pro (gemini-3.1-pro-preview) through the gemini API." 2026. Available: https://ai.google.dev/

[27] L. Li *et al.*, "SegviGen: Repurposing 3D generative model for part segmentation," *arXiv preprint arXiv:2603.16869*, 2026.

[28] OpenMOSS-Team, "MOSS-SoundEffect v2.0." Hugging Face, 2026. Available: https://huggingface.co/OpenMOSS-Team/MOSS-SoundEffect-v2.0

[29] Y. Wu, K. Chen, T. Zhang, *et al.*, "Large-scale contrastive language-audio pretraining with feature fusion and keyword-to-caption augmentation," *arXiv preprint arXiv:2211.06687*, 2022.

[30] Black Forest Labs, "FLUX.2 [klein] 4B model card." Hugging Face, 2026. Available: https://huggingface.co/black-forest-labs/FLUX.2-klein-4B

[31] D. Rempe, M. Petrovich, Y. Yuan, H. Zhang, X. B. Peng, *et al.*, "Kimodo: Scaling controllable human motion generation," *arXiv preprint arXiv:2603.15546*, 2026.

[32] Blender Foundation, "Blender." 2026. Available: https://www.blender.org/

[33] Qwen Team, "Qwen-Image-Edit-2511 model card." Hugging Face, 2025. Available: https://huggingface.co/Qwen/Qwen-Image-Edit-2511

[34] Qwen Team, "Qwen3.8-27B model card." Hugging Face, 2026. Available: https://huggingface.co/Qwen/Qwen3.8-27B

[35] O. Siméoni, H. V. Vo, M. Seitzer, *et al.*, "DINOv3," *arXiv preprint arXiv:2508.10104*, 2025.

[36] K. Deng *et al.*, "GeoSAM2: Unleashing the power of SAM2 for 3D part segmentation," *arXiv preprint arXiv:2508.14036*, 2025.

[37] L. Yang, B. Kang, Z. Huang, *et al.*, "Depth Anything V2," *arXiv preprint arXiv:2406.09414*, 2024.

[38] World Labs, "Terms of Service." 2026. Available: https://docs.worldlabs.ai/terms-of-service

[39] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, "LLM2Vec: Large language models are secretly powerful text encoders," in *Conference on language modeling (COLM)*, 2024.

[40] A. Grattafiori, A. Dubey, A. Jauhri, *et al.*, "The Llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.

[41] X. Yang, D. Kukreja, D. Pinkus, others, and K. Kitani, "SAM 3D Body: Robust full-body human mesh recovery," *arXiv preprint arXiv:2602.15989*, 2026.

[42] A. Qammaz, "SAM3DBody-cpp: A standalone C++ inference engine for SAM 3D Body." GitHub, 2026. Available: https://github.com/AmmarkoV/SAM3DBody-cpp

[43] M. Korosteleva and O. Sorkine-Hornung, "GarmentCode: Programming parametric sewing patterns," *ACM Transactions on Graphics*, vol. 42, no. 6, p. 197, 2023.

[44] The Newton Developers, "Newton: A GPU-accelerated physics simulation engine." 2026. Available: https://github.com/newton-physics/newton

[45] M. Macklin, "Warp: A high-performance Python framework for GPU simulation and graphics." NVIDIA, 2022. Available: https://github.com/NVIDIA/warp

[46] C. Ye *et al.*, "Hi3DGen: High-fidelity 3D geometry generation from images via normal bridging," in *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)*, 2025.

[47] Black Forest Labs, "FLUX.2 [klein] Base 4B model card." Hugging Face, 2026. Available: https://huggingface.co/black-forest-labs/FLUX.2-klein-base-4B

[48] thedeoxen, "refcontrol: Reference-depth LoRA for FLUX.2 [klein] 4B." Hugging Face, 2026. Available: https://huggingface.co/thedeoxen/refcontrol-FLUX.2-klein-4B-reference-depth-lora

[49] J.-P. Zhang, C.-F. Pu, M.-H. Guo, Y.-P. Cao, and S.-M. Hu, "One model to rig them all: Diverse skeleton rigging with UniRig," *ACM Transactions on Graphics (SIGGRAPH)*, 2025.

[50] R. Abdrashitov, K. Raichstat, J. Monsen, and D. Hill, "Robust skin weights transfer via weight inpainting," in *SIGGRAPH asia 2023 technical communications*, 2023.

[51] Anthropic, "Claude Code." 2026. Available: https://www.anthropic.com/claude-code

[52] H. I. I. Tam, H. I. D. Pun, A. T. Wang, A. X. Chang, and M. Savva, "SceneEval: Evaluating semantic coherence in text-conditioned 3D indoor scene synthesis," *arXiv preprint arXiv:2503.14756*, 2025.

[53] H. Xia *et al.*, "SAGE: Scalable agentic 3D scene generation for embodied AI," *arXiv preprint arXiv:2602.10116*, 2026.

Appendix

A The hub in six rounds

The surface library, one model per real-world object and the straightness check came out of six rounds of work on the hub on 6 and 7 October 2026, each answering the creator's verdict on the round before (Table A1). In the first two rounds, he chose between the old and the new method side by side from the same cameras.

Table A1. The hub's six rounds. Cost is given as rented GPU time in euros and picture-model calls in dollars. Frame time is the mean frame time on an RTX 5080, in the night scene, at 1600×900.

Round	Change	Creator's verdict	Cost	Frame time
1, two walls, A/B	surface library, routing by shape, room checks	chose the new method ("Y looks much cleaner")	€0.88	7.81 ms vs 7.53
2, two walls, A/B	71-variant library, shared texture sets, roof check	chose the new method again	€1.70	8.05 ms vs 8.24
3, whole room	all stages end to end, 74 models	old models were still loaded; labels and floor fittings were wrong	€1.67	6.78 ms (old 9.22)
4, whole room	detail through the pipeline, picture colours kept	"pretty much all of this became messy"	€8.50, $5.09	9.6 ms
5, eight pieces, A/B	shape only from the pipeline, library surfaces, parts check	new method preferred for the door, notice board and porthole; three pieces still wrong	€0.21	not run
6, whole room	round 5 everywhere, one model per object, straightness check	"this looks amazing, I have no negative feedback"	€3.48, $1.07	5.6 ms

Three findings shaped the method. First, deterministic checks catch what agents miss: in round one, the share of rays leaking out through walls and roof fell from 1.53 percent of 5,817 to zero, and the colour noise within surfaces from a median of 31.1 to 3.2 percent, but only walking through the loaded room showed 26 old models still in it, which became the made-only check. Second, generated pieces that kept their pictures' colours made the room inconsistent, so surfaces come only from the library. Third, generated composite objects melted their parts together, and generated boxes leaned (one locker by 13 degrees), which led to one model per real-world object and to the straightness check. By round six the room used 0.45 million triangles, against 1.96 million in round four, and 187 MB on disk, against 487 MB.

B Check details

- **Room checks** cast rays to find light leaking out through walls and roof, find pieces in front of a real opening and roof and wall fittings more than 3 cm off their surface or turned more than 5 degrees from it, and count pieces, triangles and lamps against the room's budget.
- **Door check:** every door must stand in a wall or partition that fully separates its two sides, tested by walking a grid from one side to the other.
- **Model check:** every model must have walls at least 3 mm thick. A generated box fails if one of its sides tilts more than 2 degrees or is warped. Making a generated model solid stops if its bounding box grows more than 3 percent.
- **Likeness gate:** a made model's outline from eight sides must overlap its close-up's by at least 0.50, a least set on the hub's twelve accepted generated models with a margin of 0.05, and its DINOv2 [16] similarity must reach a least set the same way; without that calibration the gate answers unknown.
- **Finish maps:** a close-up's map passes when its outline overlaps the close-up's by at least 0.85 and it uses every listed finish; a part whose second finish holds 20 percent of its voted area mixes finishes and fails.
- **Bake check:** each baked part within ΔE 20 (CIE76) of its surface's colour and within ΔE 32 of its close-up's; lightness order fails when the close-up shows one surface more than 10 L* lighter and the bake makes it more than 3 L* darker; one look fails when every two surfaces bake within ΔE 12, near neutral, while the close-up shows them more than ΔE 24 apart; a

quadrant of a standing model's base more than 3 cm off the floor fails. Glass, screens, lamps and print are judged by their own colour and left out of the order.

- **Settling and resting check:** an object that would turn more than 15 degrees or drift more than 30 cm when dropped keeps its laid pose and is marked as not resting; small debris, with no side over 1.4 m, may turn freely and drift up to 1 m. Hung objects are exempt by their inventory anchor, and fixed objects are not judged for tipping. The triage fails a foot off its support and an up axis leaning more than 3 degrees.
- **Light:** the place's median brightness within 1.5 times its concept's and every view within 2.5 times; a lamp is blown when over 20 percent of the pixels where it gives at least half the light are white, and unlit when it gives less than 0.001 percent of every view's light.
- **Motion:** no mover within 3 cm of a still thing or another mover, read every 0.25 s; rounds join within 1 mm; speeds within each kind's cited range.
- **Sound:** CLAP score against the brief at least 0.30, the tenth percentile of 55 of the world's picks; a take no more than 0.5 s longer than its brief.
- **People:** the shell gate passes a look when 99 percent of its soft points lie within 5 cm of the fitted body; the worst point and its joint are recorded. A clip is usable when no frame tucks more than 5 mm, no hard part stretches past 1.2 times its bind length, cloth crosses cloth no more than in the earlier body's same clip, and no part's edges pass through cloth deeper than 3 mm more often than at the bind. A far-only look is judged on its joins, likeness and the far body's triangles (at most 1,220); the near outfit's triangles and shell offset are recorded, not gating. The A-pose outline is judged against the finished Pixal3D take the build starts from; the take's own loss against the raw model is recorded apart. A crowd copy stands only where it fills under a tenth of every view's height.
- **Completion gate:** the requirements given in Section 3.6, each with its result tied to a fingerprint of the stage's files and inputs; a result made on another stage counts as unknown. The placeholder check judges every mesh a code builder draws, and only the shell builders pass unnamed.

Decisions recorded in these checks. Three people's gates are decisions of the captain, the agent session that runs world 1, taken on 12 and 13 October 2026 under the creator's rules and open to his reversal. (1) The shell gate counts 99 percent of the soft points rather than every point, with the worst point recorded: the points beyond it were loose cloth edges (cuffs, hems, caps), which no part label names, so they could not be given a joint of their own by label. (2) A look seen only far away is judged on what is drawn far away, not on the near outfit it was made from. (3) The A-pose outline is judged against the finished take, not the raw model, so the take's own loss is not charged to the build. Earlier, the full-body cut (which looks get bodies in full and which only far ones) was set by the people's share of the budget.

C Models and licences

Every model in the framework must allow commercial use of its output (Table C1); we read each licence for its clauses on commercial use. Candidates were dropped under this rule, among them NVIDIA's Lyra 2.0 (weights for internal research only), Hunyuan3D 2.1 (licence does not apply in the European Union) and GarmentCode's fork of NVIDIA Warp (non-commercial research only), which Newton's cloth solver on upstream Warp replaced; Blender's cloth simulation [32] is kept as a second route. The one open doubt is the face LoRA, whose training data is not disclosed.

Table C1. Models in the framework.

Model	Role	Licence
Nano Banana Pro [24]	concepts, close-ups on the prop route and those the open model fails, ground skins	paid API
Nano Banana Pro [24]	finish maps: the close-up drawn again with one flat colour per finish	paid API
Gemini 3.1 Pro [26]	each model's finishes listed on its close-up, and their library surfaces named	paid API
Qwen-Image-Edit-2511 [33]	close-ups, first (before the prop route)	Apache-2.0
Qwen3.8-27B [34]	the close-ups' shape check; each part's material	Apache-2.0
Pixal3D [17]	picture to 3D	MIT; DINOv3 licence [35] for its encoder
PartCrafter [20]	parts	MIT; its non-commercial background remover not used
SegviGen [27]	lays the close-up's finish map onto the 3D shape (surface stage)	MIT, on TRELLIS.2 (MIT); DINOv3 licence for its encoder; its non-commercial background remover and nvdiffrast not used
GeoSAM2 [36]	parts of our own model, seeded by the close-up's regions (trial)	Apache-2.0
Depth Anything V2 Small [37]	ground relief	Apache-2.0
ProcFunc, Infinigen shaders	surfaces	BSD-3-Clause
MOSS-SoundEffect v2.0, CLAP	sound	Apache-2.0
World Labs [38]	optional world step	paid; outputs owned by paid users
FLUX.2 klein 4B [30]	earlier prop pictures; the characters' A-pose pictures from words and their hair's clay pictures	Apache-2.0
MoGe-2 [19], SAM 3 [18]	measuring and finding objects in pictures; counting people and crops on the concepts	MIT; SAM Licence
Kimodo [31]	the characters' motion from sentences	Apache-2.0 code; NVIDIA Open Model License for the Kimodo-SOMA-RP-v1.1 weights, commercial use allowed
LLM2Vec [39] on Meta-Llama-3-8B-Instruct [40]	Kimodo's text encoder, used only while clips are made; nothing from it ships	Llama 3 Community License: commercial use allowed, the 700 million monthly users clause does not apply, "Built with Meta Llama 3" shown, no exclusion in the European Union for the text-only 8B model
SAM 3D Body [41], SAM3DBody-cpp [42]	body shape from a picture	SAM Licence for the weights, SAM3DBody-cpp's converted ONNX and GGUF weights included whatever their tag says; MIT for SAM3DBody-cpp's code
SOMA-X and MHR	the characters' bodies	Apache-2.0
GarmentCode [43], Newton [44] on Warp [45], Blender's cloth [32]	the clothes' sewing patterns and their drape	MIT; Newton and Warp Apache-2.0; Blender is GPL and its output is ours; GarmentCode's Warp fork (non-commercial research only) not used
Hi3DGen [46]	the characters' hair	MIT code; MIT and Apache-2.0 weights

Model	Role	Licence
FLUX.2 klein base 4B [47] with the refcontrol reference-depth LoRA [48]	the characters' faces	Apache-2.0, both; the LoRA's training data is not disclosed
UniRig [49]	the skeleton and skin of a quadruped	MIT code and weights
Robust Skin Weights Transfer [50]	the SOMA-X skeleton's skin weights moved onto a Pixal3D person (a trial)	MIT reference code, ported with trimesh and SciPy; libigl, whose bindings carry GPL-3.0, not used
MakeHuman eyes	the characters' eyes	CC0

The close-ups' shape check was tuned on 40 close-ups that four open picture models drew of the lab's ten objects, each scored by hand, and on Nano Banana Pro's ten pictures of the same objects. With the majority of three sampled answers, the check agreed with the hand score on 29 of the 40, passed all ten of the closed model's pictures and two marginal ones, passed none scored wrong or failed, and failed nine that were good or usable. The judge's single answer at temperature zero was rejected: asked the same 51 questions twice in one batch, it changed its verdict on 7. On the lab, 7 of the closed model's first pictures failed for drawing the room or dimension lines around the object, and all 7 passed when redrawn from the crop alone. Table C2 gives the stage's runs on world 1.

Table C2. The close-up stage on world 1. Cloud is rented card time in euros; the closed model's pictures are in dollars.

Run	Close-ups	Open model accepted	Closed model accepted	Left for the creator	Cloud	Closed model
Lab, 22 rows (earlier check: one answer)	22	8	14	0	€4.12	$2.81
World 1's remaining rows (three places)	17	0	10	7	€3.06	$4.02

The picture-to-3D step, the part splitting, the surface bakes, the sound generation, all checks and the assembly run automatically. Coding agents (Claude Code [51]) write the plans, the inventories and every code builder; in the hub's final round, 86 of its 98 models were built in code. Some steps were still done by hand: some generated objects were turned to face the right way by eye, and a few labels were placed by hand before a rule took over that job. The concepts and the verdicts are the creator's own choices.

D The surface library

The surface library holds families of variants, among them painted, steel, aluminium, rubber, plastic, cable, fabric, composite, glass, screen, light, print, wood, ground, foil, water, skin and hair. After the hub's second round it held 71 variants made from 14 ProcFunc recipes; it now holds 369 variants in 26 families made from 22 recipes. Each variant is a recipe with settings, takes its colours only from the palette, and is baked into base-colour, metal-roughness and normal maps at a texture density that depends on how close the player can get. Wear has three levels and comes from causes: edges, and kicks within 32 cm of the floor that a piece stands on. A surface the finish-map route proposes enters an append-only registry and joins the library only when the creator accepts it.

E With and without the agent tools

The tools that check an agent's work were tested on three places of world 1: a room (the workshop, 814 objects), an outdoor place (the street, 1,198 objects) and a dense room (the hangar, 1,657 objects). They are the completion gate (Section 3.6); a triage of the resting check, which judges each object by what holds it up, read from its inventory row, and keeps only real faults [52], [53]; a pick lock, which refuses an export that would show anything but the model the creator picked for a row; and a render-and-compare, which lists the objects least like their close-ups by DINOv2 similarity [16]. In both arms, the same agent (Claude Opus 5.5 in Claude Code, headless) started from the same exported stage and inventory and was asked to make the place as complete as it could in about 50 minutes, through its layout data only; one arm had the framework from before these tools. Each arm ran once, so Table E1 shows what happened, not an average.

Table E1. One session per arm and place. Faults are the resting triage's real faults left on the final stage, measured against the arm's own data; placeholders by the first placeholder check; rows are inventory rows with no object and no written reason. Agent cost is the session's model cost; cloud is its rented machines' cost.

Place, arm	Faults, start → end	Placeholders	Rows missing	Min.	Agent cost	Output tokens	Cloud	Exports	Unknown gate calls
Workshop, without	18 → 13	0 → 0	1 → 0	36	$2.26	30.4k	€0.85	7	not used
Workshop, with	18 → 0	0 → 0	1 → 0	26	$1.42	14.3k	€0.85	2	2 of 4
Street, without	11 → 0	0 → 0	0 → 0	38	$2.40	30.0k	€0.32	9	not used
Street, with	12 → 0	0 → 0	0 → 0	39	$1.21	10.4k	€0.76	1	0 of 5
Hangar, without	14 → 4	5 → 0	0 → 0	47	$2.01	26.6k	€0.00	7	not used
Hangar, with	14 → 9	5 → 0	0 → 0	32	$2.34	29.1k	€0.64	5	3 of 6

The tools made building cheaper more often than they made the scene better. On the workshop and the street, the arm with the tools used about half the agent's output and cost, and exported the stage once or twice instead of seven to nine times, because the triage named the few real faults among hundreds of alarms. On the workshop, 13 of its 18 faults were cleared by writing into the inventory what holds a piece up, which the creator still has to confirm. On the hangar the arm without the tools left fewer faults: the arm with them spent its time replacing five placeholder slabs and trying the drop test, which failed on a room with no ground mesh (fixed after the run), and the nine faults it left are models with stray parts below their feet. No agent claimed a place was complete, so the gate's hook never had to refuse a claim. No row carries an owner's pick, so the pick lock was not tested.

F World 1, place by place

Gap: Figure F1, every place of world 1. For each of the 18 places (the flat, stairwell, street, square and launch; the hub, lab, greenhouse, workshop, airlock, walkway tube, habitat, garage, hangar, wreck and old station; the camp and its grounds), the concept beside the final stage rendered from the concept's own camera. It waits on each place's one assembly passing its completion gate.